

NXTWAVE SEMICONDUCTOR

Project Technical Handover

Complete overview of the website architecture, technology, features, configuration, deployment, and maintenance workflow.

Project type: A responsive corporate semiconductor engineering website with informational pages, capabilities pages, Careers role listings, job application email delivery, contact forms, and Vercel deployment support.

1. Technology Stack

Technology	Use in this project
Next.js 16.3.3	Application framework, App Router, static pages, dynamic role pages, and API route handlers.
React 19	UI components, interactive filters, forms, animation/reveal behavior, and client-side state.
TypeScript 5.7	Type safety across pages, components, data models, and API code.
Tailwind CSS 4	Responsive layout, colors, typography, spacing, borders, shadows, and state styling.
PostCSS	CSS processing through the Tailwind PostCSS integration.
Lucide React	Interface icons used across navigation, buttons, forms, Careers, and capability sections.
Resend SDK	Server-side career application email delivery with resume attachments.
Vercel Analytics	Analytics loaded in production from the root layout.
Vercel	Recommended hosting and deployment platform for the Next.js application.

2. Project Structure

app/	
page.tsx	Homepage
layout.tsx	Root metadata, fonts, viewport, analytics
globals.css	Global Tailwind/CSS and animations
about/page.tsx	About page
capabilities/page.tsx	Capabilities overview
capabilities/<service>/page.tsx	Individual capability pages
careers/page.tsx	Careers listing and filters
careers/[slug]/page.tsx	Dynamic job detail and application page
contact/page.tsx	Contact page
api/careers/route.ts	Resend career application API
components/	

site-header.tsx	Main navigation and mobile navigation
site-footer.tsx	Footer, contact details, newsletter area
page-hero.tsx	Shared inner-page hero
section-label.tsx	Shared section eyebrow label
reveal.tsx	Scroll/reveal animation wrapper
circuit-pattern.tsx	Decorative circuit SVG background
application-form.tsx	Career application form
career-role-card.tsx	Career listing card
career-role-filters.tsx	Careers service and experience filters
contact-form.tsx	Contact form UI
home/	Homepage sections
ui/	Shared button and form field primitives
data/	
content.ts	Homepage strengths, capabilities, process data
jobs.ts	Career role data
lib/	
site.ts	Navigation and service link configuration
utils.ts	Shared utility functions
public/	Static logos, photos, technical images, and icons
docs/	Handover documentation

3. Website Routes

Route	Purpose	Rendering
/	Homepage with hero, mission, strengths, services, and process sections.	Static
/about	Company overview and semiconductor expertise.	Static
/capabilities	Complete services/capabilities overview.	Static
/capabilities/<service>	Individual service detail pages such as RTL, DFT, physical design, and validation.	Static
/careers	Open role listing with service and experience filters.	Static
/careers/<slug>	Individual role detail page with application form.	Static generated paths
/contact	Contact information and contact form.	Static
/api/careers	Multipart career application endpoint that sends email through Resend.	Dynamic server route

4. Visual System and Frontend Design

- Primary visual palette uses navy, brand blue, white, pale blue, and cyan accents.
- Typography uses Next Font with Inter for sans-serif interface text and Source Serif 4 for editorial headings.
- Layouts are responsive using Tailwind breakpoints and grid/flex composition.

- Decorative engineering visuals include circuit grids, circuit traces, pulsing nodes, animated marquee content, and reveal transitions.
- Reduced-motion support is included for the primary CSS animations.
- Navigation includes desktop links, Services menu behavior, and a mobile menu.
- Shared components keep visual behavior consistent across About, Capabilities, Careers, and Contact pages.

5. Homepage Composition

The homepage is assembled in `app/page.tsx` from these sections:

1. **Hero:** Main positioning statement, service CTA, contact CTA, logo visual, and announcement ticker.
2. **Mission:** Company purpose and engineering message.
3. **Core Strengths:** Four strength panels supported by public image assets.
4. **Engineering Spectrum:** Service cards and expandable service list.
5. **Process:** Engineering process and Careers/contact CTA.

6. Capabilities System

Service navigation is centralized in `lib/site.ts`. Capability content is represented in both shared data and service-specific pages.

When adding or renaming a service, update the related navigation and data references together. Existing service IDs are used by navigation, Careers filters, links, and anchors.

Important service IDs include:

```
rtl-design-microarchitecture
design-verification
design-for-test
physical-design
physical-verification
analog-mixed-signal
silicon-validation
```

7. Careers System

Career roles are managed in `data/jobs.ts`. The role type includes:

- Unique `slug`.
- Title, department, domain, location, work model, and employment type.
- Experience level and years of experience.
- Overview, responsibilities, required qualifications, preferred qualifications, skills, and benefits.
- `serviceId`, which connects the role to the service filter.

The dynamic page `app/careers/[slug]/page.tsx` uses `generateStaticParams` and `getJob(slug)`. Adding a role requires a deployment for the new page to be generated.

8. Career Application and Resend Email

The client component `components/application-form.tsx` collects candidate information and submits a `FormData` request to `/api/careers`.

The server route:

- Reads multipart form data.
- Validates required candidate fields and email format.
- Validates PDF, DOC, and DOCX file signatures.
- Rejects resumes larger than 10 MB.
- Sanitizes the attachment filename and email HTML.
- Encodes the resume as base64 for Resend.
- Sends from the verified sender and replies to the candidate.
- Returns success only after Resend returns a valid email ID.
- Logs only safe diagnostic information on the server.

Required environment variables

```
RESEND_API_KEY=your_private_resend_api_key
RESEND_FROM_EMAIL=careers@nxtwavesemi.com
CAREERS_RECIPIENT_EMAIL=info@nxtwavesemiconductor.com
```

Never commit secrets. Use `.env.local` for local development and Vercel Environment Variables for Preview or Production. The committed `.env.example` contains placeholders only.

9. Contact and Newsletter Areas

The Contact page uses `components/contact-form.tsx` for the existing contact experience. The footer contains a newsletter interface and site contact details. Any external form provider or mailbox integration for the contact/newsletter areas should be configured separately from the Careers Resend route.

10. Configuration Files

File	Purpose
<code>package.json</code>	Dependencies and scripts: <code>pnpm dev</code> , <code>pnpm build</code> , and <code>pnpm start</code> .
<code>pnpm-lock.yaml</code>	Locked dependency versions for reproducible installs.
<code>next.config.mjs</code>	Next configuration, image behavior, build settings, and dev indicators.
<code>tsconfig.json</code>	Strict TypeScript, bundler module resolution, path alias, and Next plugin.
<code>postcss.config.mjs</code>	Tailwind/PostCSS processing configuration.
<code>components.json</code>	UI component tooling configuration.
<code>.gitignore</code>	Ignores <code>.env*.local</code> , dependencies, build output, and local tooling files.

11. Local Development

```
pnpm install
pnpm dev
```

Open `http://localhost:3000`. For career email testing, create the private `.env.local` file before starting the server.

Useful checks:

```
pnpm exec tsc --noEmit
pnpm build
pnpm start
```

12. Vercel Deployment

1. Connect the repository to the correct Vercel project.
2. Use the project root as the deployment root.
3. Configure the three server environment variables in Vercel.
4. Configure Production and Preview separately when required.
5. Deploy through the approved branch or pull request workflow.
6. Redeploy after changing any environment variable.
7. Check the deployment logs and test the live routes.

Vercel automatically runs the Next.js build using the project scripts. The production output includes the dynamic `/api/careers` route.

13. Maintenance Workflow

1. Create a feature branch for each change.
2. Keep content changes separate from code or design changes.
3. Run TypeScript validation and production build locally.
4. Test desktop and mobile layouts.
5. Test Careers role links, filters, and application behavior after role updates.
6. Review environment changes carefully and never commit secret files.
7. Open a pull request, review the Vercel Preview, then deploy to Production.

14. Important Technical Notes

- Do not edit generated folders such as `.next` or generated TypeScript metadata.
- Do not rename environment variables without updating the API route and deployment configuration.
- Do not change a job slug casually because it changes the public URL.
- Resend acceptance means the provider accepted the email request; it does not guarantee inbox delivery.
- Use Resend logs to investigate queued, sent, delivered, bounced, failed, or suppressed messages.

- Check spam, quarantine, mailbox rules, and recipient mail filtering when Resend reports delivery but the recipient cannot find the message.

NXTwave Semiconductor | Project Technical Handover | Keep private environment values out of source control.